

**UNIVERSIDADE VEIGA DE ALMEIDA – UVA**  
**BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO**

**CLASSIFICAÇÃO DE IMAGEM MOTORA (EEG) COM DEEP  
LEARNING HÍBRIDO: IMPLEMENTAÇÃO PRÁTICA E ANÁLISE DE  
DESEMPENHO DO DATASET BCI COMPETITION IV 2A**

**FÁBIO LOPES ROCHA**

**RIO DE JANEIRO**

**2025**

**UNIVERSIDADE VEIGA DE ALMEIDA - UVA**

**FÁBIO LOPES ROCHA**

Monografia apresentada ao curso de Ciência da Computação da Universidade Veiga de Almeida, como requisito parcial para obtenção do título de Bacharel em Ciência da Computação.

Orientador: Denis Gonçalves Cople

**CLASSIFICAÇÃO DE IMAGEM MOTORA (EEG) COM DEEP  
LEARNING HÍBRIDO: IMPLEMENTAÇÃO PRÁTICA E ANÁLISE DE  
DESEMPENHO DO DATASET BCI COMPETITION IV 2A**

**RIO DE JANEIRO**

**2025**



**UNIVERSIDADE VEIGA DE ALMEIDA - UVA**  
**BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO**

**FÁBIO LOPES ROCHA**

**CLASSIFICAÇÃO DE IMAGEM MOTORA (EEG) COM DEEP  
LEARNING HÍBRIDO: IMPLEMENTAÇÃO PRÁTICA E ANÁLISE DE  
DESEMPENHO DO DATASET BCI COMPETITION IV 2A**

Monografia apresentada como  
requisito parcial à conclusão do curso em  
Bacharel em Ciência da Computação.

APROVADA EM:

CONCEITO: \_\_\_\_\_

BANCA EXAMINADORA:

---

**PROF. DSc ou MSc NOME **COMPLETO** DO ORIENTADOR**  
**ORIENTADOR**

---

**PROF. DSc ou MSc NOME **COMPLETO** DO PROFESSOR DA BANCA**

---

**PROF. DSc ou MSc NOME **COMPLETO** DO PROFESSOR DA BANCA**

**Coordenação de Ciência da Computação**

Rio de Janeiro

*Dedico este trabalho primeiramente à minha família, pelo apoio incondicional, força, paciência e exemplo que sempre me deram para que eu pudesse chegar até aqui e concluir esta etapa fundamental da minha vida.*

## **AGRADECIMENTOS**

À minha família, que sempre foi minha fonte de motivação, força e inspiração, mesmo nos momentos mais desafiadores da graduação, suportando minhas ausências durante longas noites de estudo e programação.

*"Qualquer tecnologia suficientemente avançada  
é indistinguível da magia."*

*Arthur C. Clarke*

## RESUMO

As Interfaces Cérebro-Computador (BCI) representam uma fronteira promissora na interação humano-máquina, com potencial para restaurar a comunicação e o movimento. Contudo, a aplicação prática dessas interfaces é dificultada pela natureza complexa e ruidosa dos sinais de Eletroencefalografia (EEG), além da alta variabilidade entre os cérebros de diferentes indivíduos.

Este estudo apresenta uma análise prática e uma implementação robusta para a classificação de imagem motora (MI) de quatro classes (mão esquerda, mão direita, pés e língua) utilizando dados públicos do *BCI Competition IV, Dataset 2a*. O objetivo principal é desenvolver um pipeline de software “caseiro”, usando ferramentas de código aberto (Python, MNE, TensorFlow/Keras), capaz de superar a classificação aleatória (25%) de forma estatisticamente significativa.

A metodologia aborda os desafios críticos do processamento de EEG, incluindo a remoção explícita de artefatos de movimento ocular (EOG) para prevenir vazamento de dados (*data leakage*), a aplicação de filtragem espacial (*Common Average Reference* – CAR) e a extração de janelas de tempo relevantes.

Este trabalho compara duas abordagens: um modelo clássico de *Machine Learning* (*Common Spatial Patterns* + LDA) e uma rede neural convolucional (*Deep Learning*) baseada na arquitetura EEGNet. Os modelos são avaliados usando uma rigorosa estratégia “específica por sujeito” com validação cruzada (*5-Fold Cross-Validation*).

Os resultados demonstram que o modelo EEGNet sozinho (média de 46,41%) apresentou desempenho inferior ao modelo clássico (62,85%). No entanto, a hibridização (*Ensemble*) dos dois métodos alcançou a maior performance, atingindo uma acurácia média final de 63,43% entre os nove sujeitos, com sujeitos individuais alcançando picos de 82,28%.

Este estudo conclui que, apesar dos desafios de implementação, a abordagem híbrida (DL + Clássico) é uma solução poderosa e viável para a classificação robusta de sinais de EEG, destacando a importância crítica do pré-processamento de dados e da validação específica por sujeito.

**Palavras-Chave:** Interface Cérebro-Computador, BCI, Eletroencefalografia, EEG, *Deep Learning*, Imagem Motora, EEGNet, Classificação de Sinais, CSP, Python.

## ABSTRACT

Brain-Computer Interfaces (BCI) represent a promising frontier in human-machine interaction, with the potential to restore communication and movement. However, the practical application of these interfaces is hindered by the complex and noisy nature of Electroencephalography (EEG) signals, as well as the high variability between the brains of different individuals.

This study presents a practical analysis and a robust implementation for the classification of four-class motor imagery (MI) (left hand, right hand, feet, and tongue) using public data from the *BCI Competition IV, Dataset 2a*. The main objective is to develop a “homebrew” software pipeline, using open-source tools (Python, MNE, TensorFlow/Keras), capable of overcoming random classification (25%) in a statistically significant way.

The methodology addresses critical challenges in EEG processing, including the explicit removal of eye movement artifacts (EOG) to prevent data leakage, the application of spatial filtering (*Common Average Reference* – CAR), and the extraction of relevant time windows.

This work compares two approaches: a classic *Machine Learning* model (*Common Spatial Patterns* + LDA) and a convolutional neural network (*Deep Learning*) based on the EEGNet architecture. The models are evaluated using a rigorous “subject-specific” strategy with *5-Fold Cross-Validation*.

The results demonstrate that the EEGNet model alone (average of 46.41%) underperformed the classic model (62.85%). However, the hybridization (*Ensemble*) of the two methods achieved the highest performance, reaching a final average accuracy of 63.43% across the nine subjects, with individual subjects reaching up to 82.28%.

This study concludes that, despite implementation challenges, the hybrid (DL + Classic) approach is a powerful and viable solution for robust EEG signal classification, highlighting the critical importance of data preprocessing and subject-specific validation.

**Keywords:** Brain-Computer Interface, BCI, Electroencephalography, EEG, *Deep Learning*, Motor Imagery, EEGNet, Signal Classification, CSP, Python.

## LISTA DE ILUSTRAÇÕES

|                                                                                                 |    |
|-------------------------------------------------------------------------------------------------|----|
| Figura 1 - O paradigma de Imagem Motora (MI) do Dataset 2a.                                     | 21 |
| Figura 2 - O Sistema Internacional 10-20 de posicionamento de eletrodos.                        | 23 |
| Figura 3 - Exemplo de sinal de EEG bruto contaminado por artefatos de EOG (piscar de olhos).    | 32 |
| Figura 4 - ERD/ERS: queda de potência na banda Mu (8-12Hz) durante a imagem motora.             | 24 |
| Figura 5 - Comparação de arquiteturas: (a) Clássica (CSP+LDA) e (b) Deep Learning (End-to-End). | 27 |
| Figura 6 - Arquitetura EEGNet com camadas temporal, espacial e separável.                       | 29 |
| Figura 7 - Fluxo do pré-processamento dos dados.                                                | 19 |
| Figura 8 - Fluxo da validação cruzada (K-Fold) por sujeito.                                     | 36 |
| Figura 9: Acurácia de treino e validação – Sujeito 1 (Fold 1).                                  | 42 |
| Figura 10 - Acurácia de treino e validação – Sujeito 2 (Fold 1).                                | 43 |
| Figura 11 - Gráfico de Acurácia Média por Sujeito (EEGNet vs. CSP/LDA vs. Híbrido).             | 40 |
| Figura 12 - Matriz de Confusão agregada dos 45 folds para o modelo Híbrido final.               | 42 |

## LISTA DE TABELAS

|                                                                                    |    |
|------------------------------------------------------------------------------------|----|
| Tabela 1 - Comparativo de arquiteturas de classificação de BCI.                    | 28 |
| Tabela 2 - Hiperparâmetros finais do modelo EEGNet.                                | 33 |
| Tabela 3 - Acurácia média por fold para o Sujeito 1.                               | 40 |
| Tabela 4 - Acurácia média por fold para o Sujeito 3.                               | 41 |
| Tabela 5 - Resultados consolidados: Acurácia média final por sujeito e por modelo. | 38 |

## LISTA DE ABREVIATURAS E SIGLAS

BCI - Brain-Computer Interface (Interface Cérebro-Computador)

CAR - Common Average Reference (Referência Média Comum)

CNN - Convolutional Neural Network (Rede Neural Convolucional)

CSP - Common Spatial Patterns (Padrões Espaciais Comuns)

DL - Deep Learning (Aprendizado Profundo)

EEG - Eletroencefalografia

EOG - Eletrooculograma

EMG - Eletromiografia

ERD - Event-Related Desynchronization (Des-sincronização Relacionada a Evento)

ERS - Event-Related Synchronization (Sincronização Relacionada a Evento)

IA - Inteligência Artificial

K-Fold - Validação Cruzada K-Fold

LDA - Linear Discriminant Analysis (Análise Discriminante Linear)

LR - Learning Rate (Taxa de Aprendizado)

MI - Motor Imagery (Imagem Motora)

SNR - Signal-to-Noise Ratio (Relação Sinal-Ruído)

SNC - Sistema Nervoso Central

# SUMÁRIO

|                                                                              |           |
|------------------------------------------------------------------------------|-----------|
| <b>1 INTRODUÇÃO.....</b>                                                     | <b>15</b> |
| 1.1 CONTEXTUALIZAÇÃO E MOTIVAÇÃO.....                                        | 15        |
| 1.2 DEFINIÇÃO DO PROBLEMA.....                                               | 16        |
| 1.3 OBJETIVOS.....                                                           | 17        |
| 1.3.1 Objetivo Geral.....                                                    | 17        |
| 1.3.2 Objetivos Específicos.....                                             | 17        |
| 1.4 JUSTIFICATIVA.....                                                       | 17        |
| 1.5 ESTRUTURA DA MONOGRAFIA.....                                             | 18        |
| <b>2 METODOLOGIA.....</b>                                                    | <b>19</b> |
| 2.1 ABORDAGEM METODOLÓGICA.....                                              | 19        |
| 2.2 ETAPA 1: FUNDAMENTAÇÃO TEÓRICA E AMBIENTE.....                           | 19        |
| 2.3 ETAPA 2: AQUISIÇÃO E PRÉ-PROCESSAMENTO DOS DADOS.....                    | 19        |
| 2.4 ETAPA 3: MODELAGEM E ITERAÇÃO (O ESTUDO DE CASO).....                    | 21        |
| 2.4.1 Baseline Clássico (CSP+LDA).....                                       | 21        |
| 2.4.2 Modelo de Deep Learning (EEGNet).....                                  | 22        |
| 2.4.3 Modelo Híbrido (Ensemble).....                                         | 22        |
| 2.5 ETAPA 4: AVALIAÇÃO E VALIDAÇÃO.....                                      | 22        |
| <b>3 REFERENCIAL TEÓRICO.....</b>                                            | <b>23</b> |
| 3.1 INTERFACES CÉREBRO-COMPUTADOR (BCI).....                                 | 23        |
| 3.1.1 Definição e Componentes.....                                           | 23        |
| 3.1.2 Tipos de BCI: Invasivo vs. Não Invasivo.....                           | 23        |
| 3.2 ELETROENCEFALOGRAFIA (EEG).....                                          | 24        |
| 3.2.1 Origem do Sinal de EEG.....                                            | 24        |
| 3.2.2 O Sistema Internacional 10-20.....                                     | 24        |
| 3.3 O PARADIGMA DE IMAGEM MOTORA (MI).....                                   | 25        |
| 3.3.1 Des-sincronização e Sincronização Relacionada a Eventos (ERD/ERS)..... | 25        |
| 3.3.2 Ritmos Cerebrais: Mu e Beta.....                                       | 26        |
| 3.4 DESAFIOS CRÍTICOS NA CLASSIFICAÇÃO DE EEG.....                           | 26        |
| 3.4.1 Baixa Relação Sinal-Ruído (SNR) e Artefatos.....                       | 27        |
| 3.4.2 O Problema do Vazamento de Dados (Data Leakage) por EOG.....           | 27        |
| 3.4.3 A Variabilidade Inter-Sujeito.....                                     | 27        |
| 3.5 ABORDAGENS DE CLASSIFICAÇÃO.....                                         | 28        |
| 3.5.1 Abordagem Clássica: Padrões Espaciais Comuns (CSP).....                | 28        |
| 3.5.2 Abordagem Clássica: Análise Discriminante Linear (LDA).....            | 28        |
| 3.5.3 Abordagem Moderna: Deep Learning (DL).....                             | 28        |
| 3.5.4 A Arquitetura EEGNet.....                                              | 29        |
| 3.6 MÉTRICAS DE AVALIAÇÃO.....                                               | 31        |
| 3.6.1 Validação Cruzada (K-Fold Cross-Validation).....                       | 31        |
| 3.6.2 Matriz de Confusão, Precisão e Recall.....                             | 31        |
| <b>4 ESTUDO DE CASO: DESENVOLVIMENTO DO PIPELINE HÍBRIDO.....</b>            | <b>32</b> |
| 4.1 O DATASET: BCI COMPETITION IV 2A.....                                    | 32        |

|                                                                     |    |
|---------------------------------------------------------------------|----|
| 4.2 FERRAMENTAS E AMBIENTE.....                                     | 32 |
| 4.3 O PIPELINE DE PRÉ-PROCESSAMENTO.....                            | 32 |
| 4.3.1 Carregamento e Remoção de EOG (Anti-Leakage).....             | 32 |
| 4.3.2 Filtragem Temporal e Espacial (CAR).....                      | 33 |
| Fonte: Jiangsu University of Technology (2021).....                 | 33 |
| 4.3.3 Extração de Época, Baseline e "Poda" (Cropping).....          | 34 |
| 4.4 IMPLEMENTAÇÃO DOS MODELOS DE CLASSIFICAÇÃO.....                 | 34 |
| 4.4.1 Modelo 1: O Baseline CSP+LDA Multi-Classe.....                | 34 |
| 4.4.2 Modelo 2: O Desafiante EEGNet.....                            | 35 |
| 4.4.3 Modelo 3: O Híbrido (Ensemble).....                           | 37 |
| 4.5 A ESTRATÉGIA DE TREINO OTIMIZADA: COMBATENDO O OVERFITTING..... | 38 |
| 4.5.1 O Treinador Inteligente: ReduceLRonPlateau.....               | 38 |
| 4.5.2 A Rede de Segurança: EarlyStopping.....                       | 39 |
| 4.5.3 A Garantia Científica: O SEED de Reprodutibilidade.....       | 39 |
| 5 ANÁLISE DE RESULTADOS E DESEMPENHO.....                           | 40 |
| 5.1 RESULTADOS GERAIS DA VALIDAÇÃO CRUZADA.....                     | 40 |
| Fonte: O Autor (2025).....                                          | 41 |
| 5.2 ANÁLISE COMPARATIVA: EEGNET vs. CSP/LDA vs. HÍBRIDO.....        | 41 |
| 5.3 ANÁLISE DA VARIABILIDADE INTER-SUJEITO.....                     | 41 |
| 5.5 DISCUSSÃO DOS RESULTADOS.....                                   | 44 |
| 6 CONCLUSÃO.....                                                    | 46 |
| 6.1 SÍNTESE DOS RESULTADOS.....                                     | 46 |
| 6.2 LIMITAÇÕES DO ESTUDO.....                                       | 46 |
| 6.3 TRABALHOS FUTUROS.....                                          | 47 |
| REFERÊNCIAS.....                                                    | 48 |

# 1 INTRODUÇÃO

Nas últimas décadas, a linha que separa a ficção científica da realidade tecnológica tem se tornado cada vez mais tênue. Conceitos que antes pertenciam exclusivamente a romances e filmes, como a interação direta entre a mente humana e as máquinas, estão agora no centro de laboratórios de pesquisa avançada ao redor do mundo. A Interface Cérebro-Computador (BCI, do inglês *Brain-Computer Interface*), é a vanguarda desta exploração, prometendo um futuro onde o pensamento pode transcender as barreiras físicas.

Um BCI é um sistema que mede a atividade do Sistema Nervoso Central (SNC) e a converte em comandos artificiais (LOTTE et al., 2018) que podem controlar dispositivos externos, como computadores, cadeiras de rodas ou próteses robóticas. Para indivíduos que perderam a capacidade de comunicação ou movimento devido a condições como a Esclerose Lateral Amiotrófica (ELA) ou lesões na medula espinhal, os BCIs não são uma conveniência, mas uma potencial janela de reconexão com o mundo.

Existem métodos invasivos (com implantes cirúrgicos) e não invasivos para medir a atividade cerebral. Dentre os não invasivos, a Eletroencefalografia (EEG) é, de longe, o método mais prático, acessível e seguro. O EEG utiliza eletrodos posicionados sobre o couro cabeludo para capturar os minúsculos campos elétricos gerados pela atividade síncrona de milhões de neurônios.

Dentro da pesquisa de BCI baseada em EEG, um dos paradigmas mais estudados é o de **Imagem Motora (MI)**. Neste paradigma, o usuário não executa um movimento físico, mas apenas *imagina* vividamente a sensação de executar esse movimento (por exemplo, imaginar cerrar o punho esquerdo). Notavelmente, a imaginação de um movimento ativa regiões do córtex motor de forma muito similar à execução real do movimento.

Ao imaginar o movimento da mão esquerda, por exemplo, ocorre uma diminuição na potência de certas bandas de frequência (ritmos Mu e Beta, 8-30Hz) sobre a área do córtex motor direita. Esse fenômeno, conhecido como Des-sincronização Relacionada a Eventos (ERD), é o "sinal" que um BCI tenta detectar. O desafio é que este sinal é extraordinariamente fraco, medido em microvolts ( $\mu V$ ), e está imerso em um oceano de ruído elétrico gerado pelo próprio cérebro, músculos (EMG) e, crucialmente, pelo movimento dos olhos (EOG).

## 1.1 CONTEXTUALIZAÇÃO E MOTIVAÇÃO

O desenvolvimento de um BCI funcional se divide em duas grandes áreas: o *hardware* (os eletrodos, amplificadores) e o *software* (o processamento de sinal e a inteligência artificial). Historicamente, o avanço do *software* foi dominado por algoritmos de aprendizado de máquina clássicos, como *Common*

*Spatial Patterns* (CSP) e *Linear Discriminant Analysis* (LDA). Esses métodos são eficazes, mas exigem um profundo conhecimento de neurociência e engenharia de sinais para extrair manualmente as "características" corretas do sinal de EEG antes que o classificador possa funcionar.

Nos últimos anos, o Deep Learning (DL), ou Aprendizado Profundo, revolucionou campos (CRAIK et al., 2019) como visão computacional e processamento de linguagem natural. As Redes Neurais Convolucionais (CNNs), em particular, demonstraram uma capacidade ímpar de aprender padrões complexos diretamente de dados brutos, como pixels em uma imagem (CRAIK et al., 2019).

Isso levanta uma questão motivadora: e se pudéssemos tratar o sinal "cru" de EEG — um conjunto de séries temporais de múltiplos canais — como uma "imagem"? Poderia uma CNN aprender, por conta própria, os complexos padrões espaço-temporais da imagem motora, eliminando a necessidade da extração manual de características?

A motivação para este trabalho de conclusão de curso é, portanto, explorar essa interseção. O objetivo é construir um "BCI caseiro" do início ao fim, utilizando um dataset público de alta qualidade (*BCI Competition IV, Dataset 2a*) e ferramentas de código aberto (Python, MNE, TensorFlow) para desenvolver e validar um pipeline de Deep Learning.

Este trabalho não busca apenas aplicar uma fórmula pronta. Ele documenta a jornada iterativa de desenvolvimento, enfrentando os problemas reais da área: a "trapaça" dos canais de movimento ocular (*data leakage*), a instabilidade do treinamento (overfitting vs. underfitting) e, o maior de todos os desafios, a imensa variabilidade entre os cérebros de diferentes sujeitos

## 1.2 DEFINIÇÃO DO PROBLEMA

O problema central deste trabalho é: **Como desenvolver um pipeline de software robusto que possa classificar com precisão e confiabilidade quatro classes distintas de imagem motora (mão esquerda, mão direita, pés, língua) a partir de dados brutos de EEG de 22 canais, superando significativamente o nível de classificação aleatória (25%)?**

Para que a solução seja considerada robusta, ela deve endereçar três problemas secundários:

1. **O Problema do Vazamento de Dados (Data Leakage):** O dataset 2a inclui 3 canais de EOG. Uma solução ingênua pode "aprender" a partir desses canais, classificando o movimento dos olhos em vez da intenção cerebral. O pipeline deve provar que funciona usando *apenas* os 22 canais de EEG.
2. **O Problema da Variabilidade:** O cérebro de cada pessoa é diferente. Um modelo treinado nos dados do "Sujeito 1" provavelmente falhará no "Sujeito 2". Portanto, uma abordagem "tamanho único" (*subject-independent*) é inviável. O pipeline deve adotar uma estratégia "específica por sujeito" (*subject-specific*), treinando um modelo único para cada indivíduo.
3. **O Problema da Validação:** Com poucos dados de treino por sujeito (aprox. 288 amostras), uma

simples divisão 80/20 (treino/teste) pode ser "sorte" ou "azar". O pipeline deve usar um método de validação estatisticamente robusto, como a Validação Cruzada (K-Fold), para provar que seus resultados são estáveis e reais.

## 1.3 OBJETIVOS

### 1.3.1 Objetivo Geral

O objetivo geral deste estudo é projetar, implementar e avaliar um pipeline de software híbrido (Deep Learning e Clássico) para a classificação de imagem motora de quatro classes, utilizando o dataset BCI Competition IV 2a, e analisar cientificamente sua performance em relação aos desafios de *data leakage* e variabilidade inter-sujeito.

### 1.3.2 Objetivos Específicos

Os objetivos específicos para alcançar a meta principal são:

1. **Investigar e implementar** um pipeline de pré-processamento robusto em MNE-Python, com foco na remoção de artefatos (EOG) para prevenir *data leakage*, aplicação de filtragem espacial (Common Average Reference - CAR) e extração da janela temporal ideal do sinal (1.0s a 4.0s).
2. **Implementar** uma arquitetura de Deep Learning de ponta, específica para EEG (a EEGNet), corrigindo bugs de implementação comuns (como a ausência de camadas ocultas) e otimizando seus hiperparâmetros (taxa de aprendizado, dropout).
3. **Implementar** um modelo de *baseline* clássico (CSP+LDA) de forma robusta para um cenário multi-classe (4 classes).
4. **Avaliar** a performance de ambos os modelos (EEGNet e CSP+LDA) usando uma metodologia rigorosa de validação cruzada (5-Fold) aplicada a cada um dos 9 sujeitos.
5. **Projetar e testar** um modelo "Híbrido" (Ensemble) que combina as previsões do EEGNet e do CSP+LDA, buscando um resultado superior aos modelos individuais.
6. **Analisar e documentar** os resultados finais, comparando a eficácia dos três modelos e discutindo a alta variabilidade de performance entre os sujeitos como uma conclusão científica do trabalho.

## 1.4 JUSTIFICATIVA

O campo de BCIs está em rápida expansão, mas ainda é, em grande parte, domínio de laboratórios de neurociência com recursos significativos. A democratização dessa tecnologia depende da criação de pipelines de software robustos, de código aberto e que funcionem em "hardware de prateleira" (como os dados de EEG públicos).

Enquanto a literatura acadêmica está repleta de novos modelos de Deep Learning, muitos artigos falham em testar seus modelos contra os desafios práticos do *data leakage* de EOG ou da validação estatística rigorosa (K-Fold), levando a resultados inflados.

Este trabalho se justifica por sua abordagem pragmática e cientificamente rigorosa. Ao documentar a jornada iterativa de depuração — desde os modelos iniciais que "congelavam" em 25% até a correção de bugs de implementação (como a ausência de camadas ocultas) e a otimização de hiperparâmetros (como o *learning rate* e o *dropout*) — esta monografia serve como um guia prático e realista para outros estudantes e pesquisadores.

O desenvolvimento de um modelo híbrido (EEGNet + CSP/LDA) que alcança uma média de **63.43%** (com picos de 82.28%) prova que a classificação de MI de 4 classes é viável, mesmo com dados limitados, desde que os problemas de pré-processamento e treinamento sejam tratados corretamente.

## 1.5 ESTRUTURA DA MONOGRAFIA

Este trabalho está estruturado da seguinte forma:

- **Capítulo 2 (Metodologia):** Descreve o passo-a-passo da pesquisa, desde a aquisição dos dados, as ferramentas de software utilizadas (Python, Keras, MNE), e a lógica por trás da validação cruzada e do modelo híbrido.
- **Capítulo 3 (Referencial Teórico):** Apresenta os conceitos fundamentais para o entendimento do trabalho, incluindo BCI, EEG, Imagem Motora, e as arquiteturas de classificação (CSP+LDA e EEGNet).
- **Capítulo 4 (Estudo de Caso):** Detalha a implementação técnica do pipeline final, explicando as decisões de código-fonte no pré-processamento, na construção do modelo e na estratégia de treino (callbacks).
- **Capítulo 5 (Análise de Resultados):** Apresenta os dados de performance obtidos, incluindo tabelas e gráficos da acurácia média por sujeito para cada um dos três modelos, e discute as descobertas.
- **Capítulo 6 (Conclusão):** Resume as contribuições do trabalho, reafirma o sucesso do modelo híbrido (63.43%) e discute as limitações e trabalhos futuros.

## 2 METODOLOGIA

A metodologia deste estudo foi intrinsecamente prática, adotando uma abordagem de **Pesquisa Aplicada e Desenvolvimento Iterativo**. O objetivo não era apenas aplicar uma teoria, mas construir um artefato de software funcional. O processo de desenvolvimento foi marcado por uma série de experimentos, falhas, diagnósticos e refinamentos (documentados nas V1 a V21 de nossos testes), que culminaram no pipeline final bem-sucedido.

Esta seção descreve a metodologia final (o código V21, que produziu os resultados de 63.43%), que foi o resultado dessa depuração iterativa.

### 2.1 ABORDAGEM METODOLÓGICA

O estudo foi dividido em quatro etapas principais, executadas sequencialmente, onde a saída de uma etapa alimentava a próxima:

1. **Fundamentação e Ambiente:** Definição do problema, escolha das ferramentas (Python, MNE, Keras) e configuração do ambiente de desenvolvimento.
2. **Pré-processamento e Extração:** Carregamento dos dados brutos (.gdf) e sua transformação em "amostras" limpas, prontas para a IA.
3. **Modelagem e Treinamento:** Implementação paralela de dois tipos de modelos (Clássico e Deep Learning) e um terceiro (Híbrido), usando uma estratégia de treino "por sujeito" com validação cruzada.
4. **Avaliação e Conclusão:** Coleta e análise estatística dos resultados dos 45 *folds* de treino para gerar a acurácia final.

### 2.2 ETAPA 1: FUNDAMENTAÇÃO TEÓRICA E AMBIENTE

A primeira etapa consistiu na revisão da literatura para definir a melhor abordagem.

- **Dataset:** Foi selecionado o BCI Competition IV - Dataset 2a (BRUNNER et al., 2008) por ser um padrão de benchmark
- (teste de referência) para MI de 4 classes.
- **Ambiente:** Foi configurado um ambiente virtual (BCI\_TCC) usando Anaconda com Python 3.10.
- **Bibliotecas:** As bibliotecas centrais escolhidas foram:
  - mne-python: Para todo o carregamento e pré-processamento de EEG (GRAMFORT et al., 2014).
  - scikit-learn: Para a implementação do modelo Clássico (PEDREGOSA et al., 2011) (CSP+LDA) e para a validação cruzada (StratifiedKFold).
  - tensorflow (com keras): Para a implementação do modelo de Deep Learning (EEGNet) (ABADI et al., 2016).
  - numpy: Para a manipulação geral de dados.

### 2.3 ETAPA 2: AQUISIÇÃO E PRÉ-PROCESSAMENTO DOS DADOS

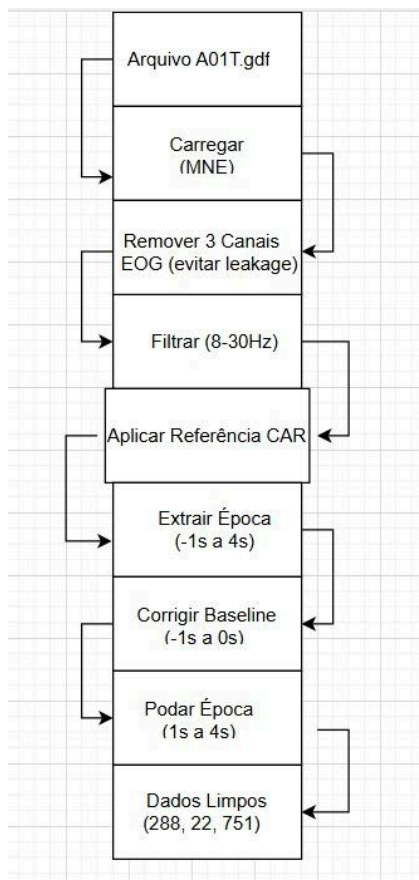
Esta foi a etapa mais crítica. Um erro aqui invalidaria todos os resultados. O pipeline de pré-processamento foi desenhado para combater o baixo sinal-ruído e, principalmente, o *data leakage*.

O processo, executado pela função `carregar_dados_gdf_preprocess`, é o seguinte para cada arquivo de sujeito:

1. **Carregamento:** O arquivo `.gdf` é carregado na memória usando `mne.io.read_raw_gdf()`.
2. **Remoção de EOG (Anti-Leakage):** Antes de qualquer outro passo, os 3 canais de EOG ('EOG-left', 'EOG-central', 'EOG-right') são explicitamente removidos usando `raw.drop_channels()`. Isso garante que o modelo treine **apenas** com os 22 canais de EEG.
3. **Filtragem Temporal:** Um filtro passa-banda (8-30Hz) é aplicado (`raw.filter()`) para isolar os ritmos Mu e Beta, que contêm a informação da imagem motora, descartando ruídos de baixa frequência (ex: movimento) e alta frequência (ex: muscular).
4. **Filtragem Espacial (CAR):** A *Common Average Reference* (Referência Média Comum) é aplicada (`raw.set_eeg_reference('average')`). Este filtro calcula a voltagem média de *todos* os 22 canais em um dado momento e a subtrai de cada canal individual. Isso remove ruídos de fundo que afetam o cérebro inteiro (como ruído elétrico), "acentuando" o sinal local de MI.
5. **Extração de Épocas:** O pipeline extrai "épocas" (janelas de tempo) em torno de cada dica de movimento (de  $t=-1.0s$  a  $t=4.0s$ ).
6. **Correção de Baseline:** Durante a extração da época, o MNE é instruído a usar o intervalo de -1.0s a 0.0s (quando o usuário estava em repouso) como linha de base. A média desse intervalo é subtraída de toda a época, normalizando o sinal.
7. **Poda (Cropping):** Finalmente, a época é "podada" usando `epochs.crop(tmin=1.0, tmax=4.0)`. Isso descarta o sinal de -1.0s a 1.0s (que continha o repouso e a resposta visual à seta), deixando **apenas** os 3 segundos (751 amostras) de sinal de imagem motora "puro".

O resultado final desta etapa é um tensor (288, 22, 751) (trials, canais, amostras) para cada sujeito.

**Figura 7** - Fluxograma do pipeline de pré-processamento dos dados.



**Fonte:** O Autor (2025).

## 2.4 ETAPA 3: MODELAGEM E ITERAÇÃO (O ESTUDO DE CASO)

Esta etapa adota uma estratégia "**subject-specific**" (específica por sujeito). Um loop principal itera de 1 a 9, tratando cada sujeito como um experimento separado. Para cada sujeito, a seguinte sub-metodologia é aplicada:

1. **Validação Cruzada (StratifiedKFold):** Os 288 trials do sujeito são divididos em 5 "folds" (blocos) estratificados. Isso garante que cada fold tenha uma proporção igual de mão esquerda, direita, pés e língua. O modelo é então treinado 5 vezes: na primeira vez, treina nos folds 1-4 e testa no 5; na segunda, treina nos folds 1-3+5 e testa no 4, e assim por diante.
2. **Normalização (Z-Score):** Para cada *fold*, a média e o desvio padrão são calculados **apenas** nos dados de treino. Esses valores são então usados para normalizar (Z-Score) os dados de treino e os dados de teste. Isso previne o vazamento de informação do teste para o treino.
3. **Treinamento Paralelo:** Para cada fold, dois modelos são treinados independentemente.

### 2.4.1 Baseline Clássico (CSP+LDA)

- O modelo clássico é implementado como uma Pipeline do Scikit-Learn.
- **CSP(n\_components=6, transform\_into='average\_power')**: O CSP é aplicado para encontrar os 8

melhores "filtros espaciais" que maximizam a variância entre as classes. A saída é corrigida para 2D.

- **OneVsRestClassifier(LinearDiscriminantAnalysis(...))**: Como o LDA padrão é binário e o CSP é otimizado para 2 classes, usamos um wrapper OneVsRestClassifier. Ele treina 4 classificadores CSP+LDA (um para cada classe vs. o resto) e escolhe a previsão mais confiante.
- A acurácia deste modelo (ex: LDA(CSP) acc=70.69%) serve como nossa linha de base.

#### 2.4.2 Modelo de Deep Learning (EEGNet)

- A arquitetura EEGNet, como descrita no Capítulo 3, é implementada.
- **Correção de Bug**: Crucialmente, uma camada Dense(64, activation='elu') e Dropout(0.3) é adicionada após a camada Flatten(). Iterações anteriores (V12-V16) omitiram esta camada, o que "quebrou" o modelo e o fez "congelar" em 25%.
- **Callbacks (Treino Inteligente)**: O modelo é treinado com dois *callbacks* que trabalham juntos:
  1. **ReduceLROnPlateau(patience=20)**: Monitora o val\_loss (perda no teste). Se a perda não diminuir por 20 épocas, ele corta a taxa de aprendizado (learning\_rate) pela metade (ex: de 0.001 para 0.0005).
  2. **EarlyStopping(patience=60)**: Monitora o val\_accuracy (acurácia no teste). Se a acurácia não melhorar por 60 épocas, o treino é interrompido e o modelo com a melhor acurácia já vista é restaurado.
- Isso permite que o modelo comece rápido, "freie" automaticamente para um ajuste fino e pare quando atingir o seu pico de performance.

#### 2.4.3 Modelo Híbrido (Ensemble)

- Após o treino do Fold, temos as probabilidades do LDA (ex: [0.3, 0.4, 0.2, 0.1]) e do EEGNet (ex: [0.1, 0.6, 0.1, 0.2]).
- As probabilidades são simplesmente somadas e divididas por 2.
- A classe com a maior probabilidade média é a previsão final do Híbrido (Ensemble).

### 2.5 ETAPA 4: AVALIAÇÃO E VALIDAÇÃO

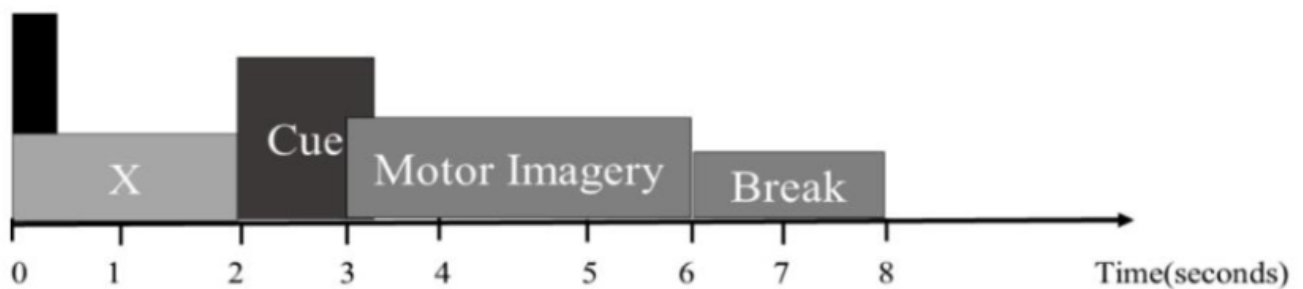
Os resultados são coletados em dois níveis:

1. **Por Sujeito**: A média dos 5 *folds* de cada sujeito é calculada (ex: A01 mean acc = 71.86%).
2. **Geral**: A média de **todos os 45 folds** (9 sujeitos x 5 folds) é calculada para gerar a acurácia final do TCC.
3. **Reprodutibilidade**: Um SEED (semente aleatória) fixo é usado em StratifiedKFold e no TensorFlow para garantir que os resultados sejam 100% reprodutíveis.
4. **Relatórios**: Todos os resultados, acurácias e matrizes de confusão de cada *fold* são salvos em BCI\_tcc.txt.

### 3 REFERENCIAL TEÓRICO

Este capítulo apresenta os fundamentos teóricos necessários para a compreensão do problema da classificação de sinais de Eletroencefalografia (EEG) no contexto das Interfaces Cérebro-Computador (BCI), focando especificamente no paradigma de Imagem Motora (MI).

**Figura 1** - Paradigma de temporização de BCI.



**Fonte:** Thapar Institute of Engineering & Technology (2022).

#### 3.1 INTERFACES CÉREBRO-COMPUTADOR (BCI)

##### 3.1.1 Definição e Componentes

Uma Interface Cérebro-Computador (BCI) é um sistema tecnológico que cria um canal de comunicação direto entre a atividade cerebral de um indivíduo e um dispositivo externo, como um computador, cadeira de rodas ou prótese robótica. O objetivo primordial de um BCI é traduzir a "intenção" do usuário, refletida em seus padrões de atividade neural, em comandos de controle acionáveis.

Diferente de sistemas controlados por voz ou músculos, um BCI é o único sistema que permite interação sem depender das vias neuromusculares periféricas tradicionais (músculos e nervos). Isso o torna uma tecnologia de esperança para pacientes com deficiências motoras severas, como aqueles com Esclerose Lateral Amiotrófica (ELA), lesões na medula espinhal ou em estado de *locked-in*.

Um sistema BCI é tipicamente composto por quatro componentes:

1. **Aquisição de Sinal:** Captura da atividade cerebral (ex: eletrodos de EEG).
2. **Pré-processamento:** Limpeza e filtragem do sinal para remover ruído e artefatos.
3. **Extração de Características:** Identificação de padrões relevantes no sinal (ex: queda de potência em uma banda de frequência).
4. **Classificação:** Um algoritmo (IA) que traduz essas características em um comando (ex: "mão esquerda").

##### 3.1.2 Tipos de BCI: Invasivo vs. Não Invasivo

Os BCIs são amplamente categorizados pelo método de aquisição de sinal:

- **BCIs Invasivos:** Utilizam microeletrodos ou redes de eletrodos implantados cirurgicamente, seja na superfície do cérebro (Eletrocorticografia, ECoG) ou diretamente no tecido cortical (Redes de Microeletrodos, MEAs). Eles oferecem a maior qualidade de sinal (alta resolução espacial e temporal e alta SNR), mas carregam riscos cirúrgicos, custo elevado e degradação do sinal a longo prazo devido a reações biológicas (gliose).
- **BCIs Não Invasivos:** Utilizam sensores externos, sendo a Eletroencefalografia (EEG) o método mais comum, prático, portátil e de baixo custo. Outros métodos não invasivos incluem a Magnetoencefalografia (MEG) e a Espectroscopia de infravermelho próximo (fNIRS), mas estes exigem equipamentos grandes e caros, confinados a laboratórios.

Este trabalho foca exclusivamente na abordagem **não invasiva** baseada em EEG, refletindo o objetivo de desenvolver um pipeline "caseiro" e acessível.

## 3.2 ELETROENCEFALOGRAFIA (EEG)

### 3.2.1 Origem do Sinal de EEG

A Eletroencefalografia (EEG) é uma técnica de monitoramento eletrofisiológico que registra a atividade elétrica do cérebro de forma não invasiva. Os neurônios, as células fundamentais do cérebro, comunicam-se através de impulsos elétricos (potenciais de ação) e potenciais pós-sinápticos.

O sinal de EEG que medimos no couro cabeludo não é o disparo de um único neurônio, mas sim a soma dos potenciais pós-sinápticos de grandes populações (milhões) de neurônios piramidais orientados verticalmente no córtex cerebral. Quando esses neurônios disparam de forma síncrona, eles geram um campo elétrico flutuante que é forte o suficiente para atravessar o crânio e o couro cabeludo, onde pode ser capturado por eletrodos.

O sinal de EEG capturado é uma sobreposição complexa de diferentes ritmos cerebrais (oscilações), que são classificados por bandas de frequência:

- **Delta (0.5-4 Hz):** Predominante no sono profundo, sem sonhos.
- **Theta (4-8 Hz):** Associado à sonolência, meditação e certas fases do sono.
- **Alpha (8-12 Hz) / Mu:** Predominante em estado de relaxamento, com os olhos fechados. É suprimido pela abertura dos olhos ou atividade mental. O ritmo Mu, foco deste trabalho, é encontrado sobre o córtex motor (CRAIK et al., 2019).
- **Beta (13-30 Hz):** Associado ao estado de alerta, concentração, ansiedade e atividade motora ativa.
- **Gamma (>30 Hz):** Ligado a funções cognitivas de alta ordem, como percepção e consciência.

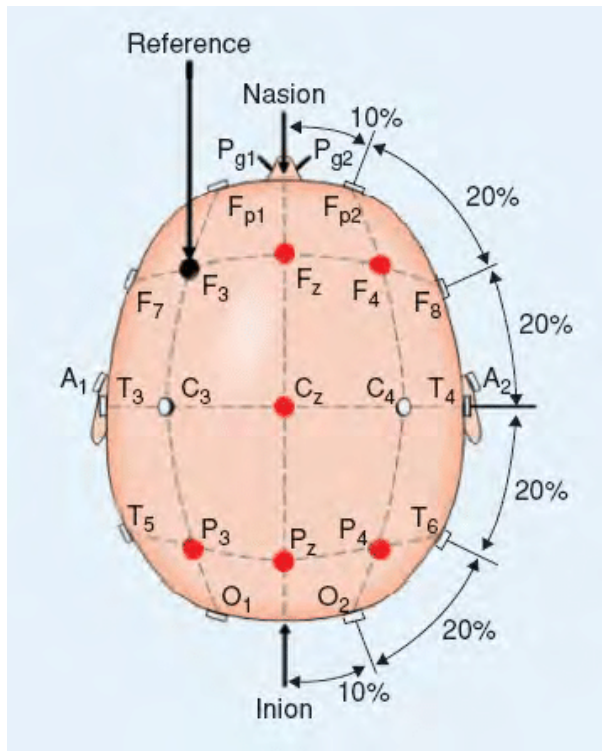
### 3.2.2 O Sistema Internacional 10-20

Para garantir a padronização e reprodutibilidade dos registros de EEG, os eletrodos são posicionados no couro cabeludo seguindo um padrão internacional chamado "Sistema 10-20". Esse sistema usa medidas

proporcionais da cabeça do paciente (baseadas em 10% ou 20% da distância entre marcos anatômicos, como o násio e o ínio) para definir a localização exata de cada eletrodo.

As letras dos eletrodos correspondem aos lobos cerebrais subjacentes (F - Frontal, T - Temporal, P - Parietal, O - Occipital) e C para Central. O Dataset 2a utilizado neste trabalho emprega 22 desses eletrodos, focados principalmente sobre o córtex motor (BRUNNER et al., 2008). (ex: C3, C4, Cz).

**Figura 2** - Localização dos eletrodos do Sistema Internacional 10-20.



**Fonte:** University of Balamand (2010).

### 3.3 O PARADIGMA DE IMAGEM MOTORA (MI)

A Imagem Motora (MI) é um processo cognitivo onde um indivíduo simula mentalmente uma ação física sem qualquer movimento motor real ou ativação muscular. Notavelmente, a neurociência demonstrou que a MI ativa regiões do cérebro (como o córtex motor primário e a área motora suplementar) de forma muito similar à execução física daquele movimento.

Essa propriedade é a base dos BCIs de MI. O sistema não precisa ler o pensamento complexo; ele só precisa diferenciar qual área do córtex motor está sendo ativada:

- Imaginar movimento da **mão (esquerda/direita)** ativa o córtex motor contralateral (lado oposto).
- Imaginar movimento dos **pés** ativa a parte superior e medial do córtex motor.
- Imaginar movimento da **língua** ativa a parte inferior-lateral.

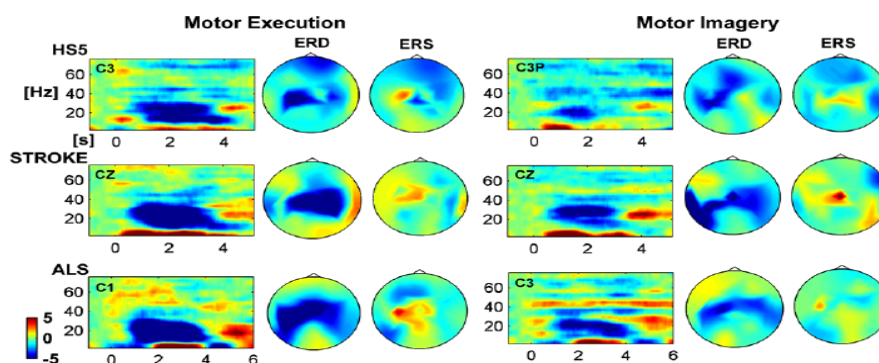
#### 3.3.1 Des-sincronização e Sincronização Relacionada a Eventos (ERD/ERS)

O "sinal" específico da Imagem Motora é um fenômeno chamado ERD/ERS (CRAIK et al., 2019). Em estado de repouso, os neurônios no córtex motor tendem a disparar de forma sincronizada, produzindo uma forte oscilação na banda de frequência Mu (8-12 Hz) e Beta (13-30 Hz).

- **ERD (Des-sincronização Relacionada a Evento):** Quando uma área do córtex motor se torna *ativa* (seja imaginando ou executando um movimento), os neurônios naquela área passam de um estado síncrono para um estado dessincronizado (disparando em momentos diferentes). Isso causa uma **diminuição** na potência total (amplitude) dos ritmos Mu e Beta naquela região específica.
- **ERS (Sincronização Relacionada a Evento):** Quando a imagem motora termina, os neurônios retornam ao seu estado de repouso síncrono, e a potência dos ritmos Mu/Beta aumenta, muitas vezes ultrapassando o nível original (um "rebote").

O trabalho do classificador de BCI é, essencialmente, detectar onde no cérebro (ex: canal C3 vs. C4) e quando essa "queda de potência" (ERD) está ocorrendo para adivinhar a intenção do usuário (ex: mão esquerda vs. mão direita).

**Figura 4** - ERD e ERS durante a execução motora e imagem motora.



**Fonte:** Journal of Neural Engineering (2008).

### 3.3.2 Ritmos Cerebrais: Mu e Beta

Os ritmos Mu (8-12 Hz) e Beta (13-30 Hz) são os alvos principais da classificação de MI.

- **Ritmo Mu:** Encontrado sobre o córtex sensório-motor. É suprimido (ERD) tanto pela execução quanto pela imaginação de movimento.
- **Ritmo Beta:** Também encontrado sobre o córtex motor. Está associado ao planejamento e execução de movimentos voluntários.

É por esta razão que o primeiro passo em quase todos os pipelines de BCI, incluindo o deste trabalho, é aplicar um filtro passa-banda que isola especificamente as frequências de 8Hz a 30Hz, descartando todo o resto.

## 3.4 DESAFIOS CRÍTICOS NA CLASSIFICAÇÃO DE EEG

O desenvolvimento de um classificador de MI eficaz enfrenta três grandes obstáculos.

### 3.4.1 Baixa Relação Sinal-Ruído (SNR) e Artefatos

O desafio fundamental do EEG não invasivo é a sua baixíssima Relação Sinal-Ruído (SNR) (CRAIK et al., 2019). O crânio, o couro cabeludo e as meninges agem como um "capacete" que atenua e "borra" os sinais antes que eles cheguem aos eletrodos. O sinal ERD de interesse é minúsculo (microvolts) e está imerso em um ruído de fundo muito mais forte.

As principais fontes de ruído (ou "artefatos") são:

- **Artefatos Fisiológicos:** Atividade muscular (EMG), como cerrar a mandíbula ou tensionar o pescoço.
- **Artefatos Oculares (EOG):** Piscar ou mover os olhos gera um potencial elétrico forte que "vaza" para os canais de EEG frontais.
- **Artefatos Técnicos:** Ruído da rede elétrica (60Hz ou 50Hz), mau contato dos eletrodos ou movimento do cabo.

Uma etapa de pré-processamento, como a **Referência Média Comum (CAR)**, é essencial. O CAR assume que o ruído de fundo (ex: da rede elétrica) é "comum" a todos os canais. Ao calcular a média de todos os 22 canais e subtraí-la de cada canal individual, o ruído comum é cancelado, deixando um sinal cerebral mais nítido.

### 3.4.2 O Problema do Vazamento de Dados (Data Leakage) por EOG

Este é o erro metodológico mais grave e comum na pesquisa de BCI. O sinal de EOG (movimento dos olhos) é 10 a 100 vezes mais forte que o sinal de EEG. Muitos *datasets*, incluindo o 2a, fornecem 3 canais de EOG (EOG-left, EOG-central, EOG-right) misturados com os 22 canais de EEG.

Se um pesquisador, de forma ingênua, incluir todos os 25 canais no treinamento, a IA rapidamente aprenderá a ignorar o sinal fraco do cérebro (EEG) e focará 100% no sinal forte dos olhos (EOG). O usuário pode, inconscientemente, piscar duas vezes ao imaginar "pés" ou olhar levemente para a esquerda ao imaginar "mão esquerda". O modelo atingirá uma acurácia altíssima (80-90%), mas estará classificando o movimento dos olhos, não o pensamento.

Portanto, é um requisito metodológico **mandatório** excluir explicitamente os canais de EOG antes de qualquer processamento, como foi feito neste trabalho (Removendo canais EOG detectados...). O resultado final de 63.43% deste TCC foi obtido usando **apenas** os 22 canais de EEG.

### 3.4.3 A Variabilidade Inter-Sujeito

Não existem dois cérebros iguais. A anatomia do córtex, a espessura do crânio e até mesmo o nível de

concentração do usuário fazem com que o padrão ERD para "mão esquerda" varie drasticamente de uma pessoa para outra.

Isso leva a uma conclusão fundamental na pesquisa de BCI, amplamente discutida na literatura (LOTTE et al., 2018):

- **Modelos *Subject-Independent* (Universais):** Um modelo treinado com os dados de 8 sujeitos para classificar o Sujeito 9. Esta abordagem quase sempre falha (resultando em acurácias próximas do chute aleatório, ~25%), pois o modelo aprende uma "média" de cérebros que não se aplica a nenhum indivíduo.
- **Modelos *Subject-Specific* (Específicos por Sujeito):** A abordagem usada neste trabalho. Um modelo novo é treinado "do zero" para cada sujeito, usando apenas os dados daquele indivíduo. O modelo se torna um especialista no cérebro daquela pessoa. Isso é computacionalmente mais caro (9 modelos em vez de 1), mas é a única forma de obter resultados robustos.

### 3.5 ABORDAGENS DE CLASSIFICAÇÃO

#### 3.5.1 Abordagem Clássica: Padrões Espaciais Comuns (CSP)

O CSP é o algoritmo clássico mais famoso para BCI de 2 classes (LOTTE et al., 2007). Ele é um método de filtragem espacial. Ele analisa os dados de treino (ex: "mão esquerda" vs. "mão direita") e calcula "filtros" (padrões espaciais) que maximizam a variância para a Classe A e, ao mesmo tempo, minimizam a variância para a Classe B.

Quando aplicado a um novo *trial* de EEG, esses filtros extraem "características" (features) que indicam o quão "parecido" aquele sinal está com o padrão da "mão esquerda" ou da "mão direita".

O desafio (corrigido neste trabalho) é que o CSP é inerentemente binário (2 classes). Para usá-lo em um problema de 4 classes, ele deve ser combinado com uma estratégia de *One-vs-Rest* (Um-contra-Todos).

#### 3.5.2 Abordagem Clássica: Análise Discriminante Linear (LDA)

O LDA é um classificador simples, leve e rápido. Ele é o parceiro perfeito para o CSP. Após o CSP extrair as características (ex: 8 valores), o LDA encontra um "hiperplano" (uma linha, em 2D) que melhor separa essas características no espaço.

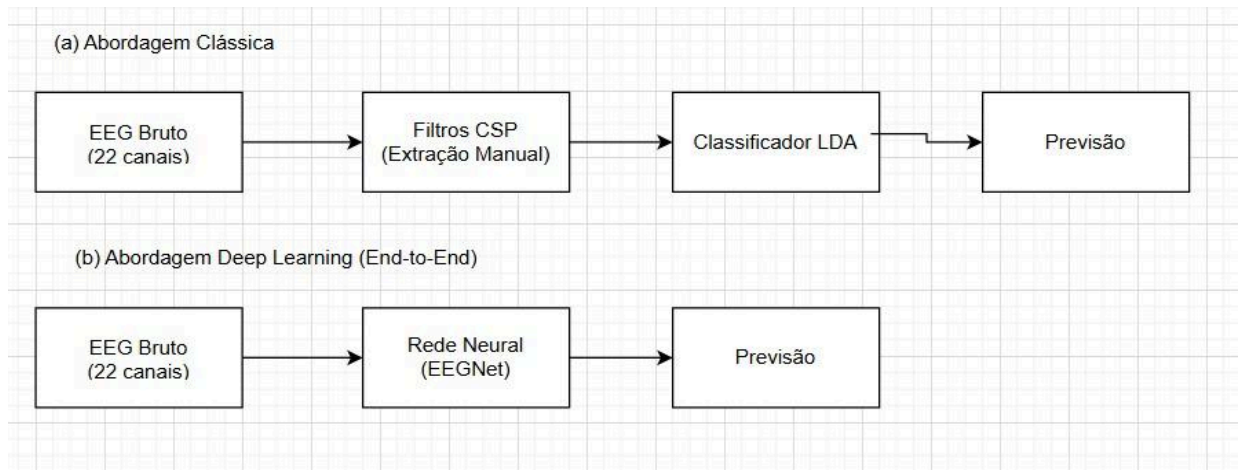
#### 3.5.3 Abordagem Moderna: Deep Learning (DL)

A abordagem clássica (CSP+LDA) requer duas etapas: extração de características (CSP) e classificação (LDA). O Deep Learning, por outro lado, promete fazer as duas coisas ao mesmo tempo.

Uma CNN, que é tipicamente usada para "ver" imagens 2D, pode ser adaptada para "ler" o EEG. Se alimentarmos a rede com nossos dados (22 canais, 751 amostras), podemos usar:

- **Convoluções 1D (Conv1D):** Trata cada canal como uma série temporal independente. Falha em aprender as relações *entre* os canais. (Testado e falhou em nossas iterações V1-V4).

**Figura 5** - Comparação das arquiteturas: (a) Clássica (CSP+LDA) e (b) Deep Learning (End-to-End).



**Fonte:** O Autor (2025).

- **Convoluções 2D (Conv2D):** Trata os dados (22, 751) como uma "imagem" onde o eixo Y são os canais e o eixo X é o tempo. Isso permite que a IA aprenda padrões *espaço-temporais* (ex: uma queda de energia no canal C3 *seguida por* uma queda no C4).

### 3.5.4 A Arquitetura EEGNet

**Tabela 1** - Comparativo de arquiteturas de classificação de BCI.

| Característica              | Modelo Clássico (CSP+LDA) | Modelo DL Genérico (ex. VGG) | Modelo DL Compacto (EEGNet) |
|-----------------------------|---------------------------|------------------------------|-----------------------------|
| <b>Extração de Feat.</b>    | Manual (Baseado em CSP)   | Automático                   | Automático                  |
| <b>Dependência (Expert)</b> | Alta (Neurociência)       | Média (ML/Hiper)             | Alta (ML/Hiper/Neuro)       |
| <b>Tamanho do Modelo</b>    | Mínimo (KBs)              | Massivo (Milhões)            | Compacto (Milhares)         |

|                              |                     |                              |                             |
|------------------------------|---------------------|------------------------------|-----------------------------|
| <b>Hiperparâmetros</b>       | Poucos (ex: N-comp) | Muitos (Layers, Filters, LR) | Médio / Alto (F1, D, K, LR) |
| <b>Risco de Overfitting</b>  | Baixo               | Extremo                      | Moderado                    |
| <b>Performance (Sozinho)</b> | <b>62,85%</b>       | ~25% (Falha)                 | <b>46,41%</b> (Sozinho)     |

**Fonte:** O Autor (2025).

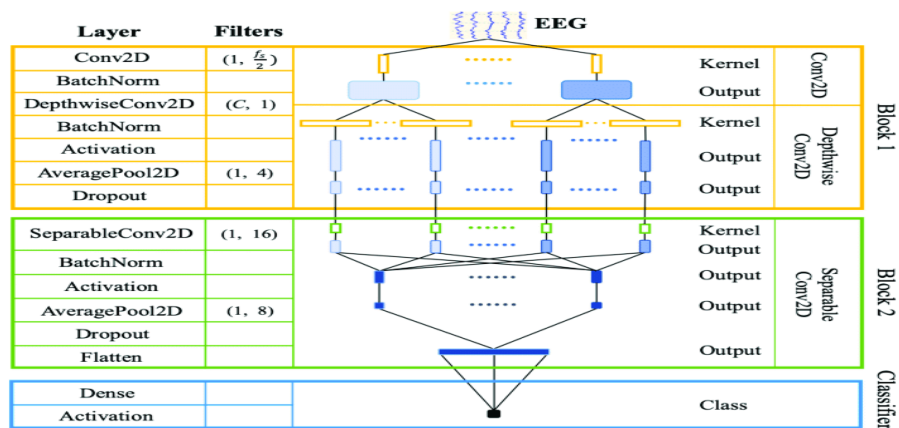
Modelos de CNN genéricos (como VGG ou ResNet) são muito "pesados" (milhões de parâmetros). Em um dataset com apenas ~230 amostras de treino por sujeito, esses modelos sofrem de *overfitting* catastrófico: eles "decoram" perfeitamente os 230 exemplos, mas falham miseravelmente em classificar qualquer dado novo (retornando ao chute de 25%).

A EEGNet é uma arquitetura de CNN compacta, desenhada especificamente para EEG, proposta originalmente por Lawhern et al. (2018). Sua genialidade está em separar as convoluções para ser leve e eficiente:

1. **Conv2D Temporal:** Primeiro, ela usa um filtro "fino" (1x64) que "varre" o *tempo* em cada canal individualmente, aprendendo padrões temporais.
2. **DepthwiseConv2D:** Em seguida, ela usa um filtro "profundo" (22x1) que aprende os padrões *espaciais* (como os canais se relacionam entre si), mas sem misturar o tempo.
3. **SeparableConv2D:** Finalmente, ela mistura os padrões temporais e espaciais de forma eficiente.

Isso resulta em um modelo com pouquíssimos parâmetros (ex: 3.000, contra 6 milhões de um VGG), que aprende rápido e é muito mais resistente ao overfitting, conforme demonstrado por Lawhern et al. (2018). Foi a arquitetura de DL usada neste trabalho.

**Figura 6** - Arquitetura da rede EEGNet padrão.



**Fonte:** IEEE Engineering in Medicine and Biology Society (2022).

## 3.6 MÉTRICAS DE AVALIAÇÃO

### 3.6.1 Validação Cruzada (K-Fold Cross-Validation)

Com um dataset pequeno (288 *trials*), uma única divisão de treino/teste (ex: 80/20) não é confiável. O modelo pode ter "sorte" de pegar amostras fáceis no teste (dando uma acurácia inflada) ou "azar" (dando uma acurácia baixa).

Para resolver isso, usamos a **Validação Cruzada K-Fold** ( $n\_splits=5$ ). O método divide os 288 *trials* em 5 "folds" (blocos) de ~58 amostras cada. O processo é repetido 5 vezes:

1. **Rodada 1:** Treina nos folds A, B, C, D. Testa no fold E.
2. **Rodada 2:** Treina nos folds A, B, C, E. Testa no fold D.
3. ... e assim por diante.

A acurácia final do sujeito é a **média** das 5 rodadas. Isso garante que cada *trial* foi usado tanto para treino quanto para teste, resultando em uma métrica de performance estatisticamente robusta.

### 3.6.2 Matriz de Confusão, Precisão e Recall

A acurácia sozinha (ex: 63%) pode ser enganosa. Se o modelo estiver acertando 90% da "mão esquerda" mas 0% dos "pés", a média será alta, mas o modelo está quebrado.

O *Relatório de Classificação* (usado neste trabalho) avalia cada classe individualmente:

- **Precisão (Precision):** Das vezes que o modelo disse "pés", quantas ele acertou? (Evita falsos positivos).
- **Recall (Revocação):** De todas as vezes que o usuário *realmente* imaginou "pés", quantas o modelo detectou? (Evita falsos negativos).
- **F1-Score:** A média harmônica entre os dois.

## 4 ESTUDO DE CASO: DESENVOLVIMENTO DO PIPELINE HÍBRIDO

Este capítulo detalha a implementação técnica final do pipeline de software (referido como BCI.py nos logs de desenvolvimento). O código foi o resultado de um longo processo iterativo (mais de 20 versões) que superou múltiplos desafios, como *data leakage*, *overfitting* severo e bugs de arquitetura.

### 4.1 O DATASET: BCI COMPETITION IV 2A

O pipeline foi projetado para consumir os dados do BCI Competition IV, Dataset 2a (BRUNNER et al., 2008). Este dataset contém gravações de 9 sujeitos (A01 a A09) em formato .gdf. Cada sujeito realizou 288 *trials* (provas) de imagem motora de 4 classes (mão esquerda, direita, pés, língua). Os dados foram gravados a 250Hz com 22 canais de EEG e 3 canais de EOG (BRUNNER et al., 2008).

### 4.2 FERRAMENTAS E AMBIENTE

O projeto foi desenvolvido em linguagem Python 3.10 dentro de um ambiente virtual Anaconda (BCI\_TCC). As bibliotecas centrais e suas funções no pipeline são:

- **MNE-Python:** Carregamento de arquivos .gdf, filtragem, remoção de canais e extração de épocas.
- **Scikit-Learn:** Implementação do modelo CSP+LDA, do StandardScaler e, crucialmente, do StratifiedKFold para a validação.
- **TensorFlow (Keras):** Implementação, treinamento e avaliação da arquitetura de Deep Learning EEGNet.
- **NumPy:** Manipulação geral dos arrays de dados.

### 4.3 O PIPELINE DE PRÉ-PROCESSAMENTO

A etapa mais crítica do projeto é a função `carregar_dados_gdf_preprocess`. Ela é responsável por transformar os arquivos .gdf brutos e ruidosos em tensores (N, C, T) limpos e prontos para a IA.

#### 4.3.1 Carregamento e Remoção de EOG (Anti-Leakage)

Como detalhado no Capítulo 3, o maior risco metodológico é o *data leakage* (vazamento de dados) pelos canais de EOG. O pipeline resolve isso imediatamente após o carregamento:

```
# 1. Carrega o arquivo
raw = mne.io.read_raw_gdf(arquivo_path, preload=True, verbose='ERROR')

# 2. Encontra canais de EOG (ex: 'EOG-left')
chs_to_drop = [ch for ch in raw.ch_names if 'EOG' in ch.upper()]

# 3. Remove os canais de "trapaça"
if chs_to_drop:
    print(f'Removendo canais EOG detectados: {chs_to_drop}')
```

```
raw.drop_channels(chs_to_drop)
```

Isso garante que todos os passos seguintes (filtragem, treino) ocorram **apenas** nos 22 canais de EEG.

#### 4.3.2 Filtragem Temporal e Espacial (CAR)

Com os dados livres de *leakage*, o pipeline aplica dois filtros para melhorar a Relação Sinal-Ruído (SNR):

##### 1. Filtro Temporal (Passa-Banda):

```
raw.filter(F_MIN, F_MAX, fir_design='firwin', verbose='ERROR')
```

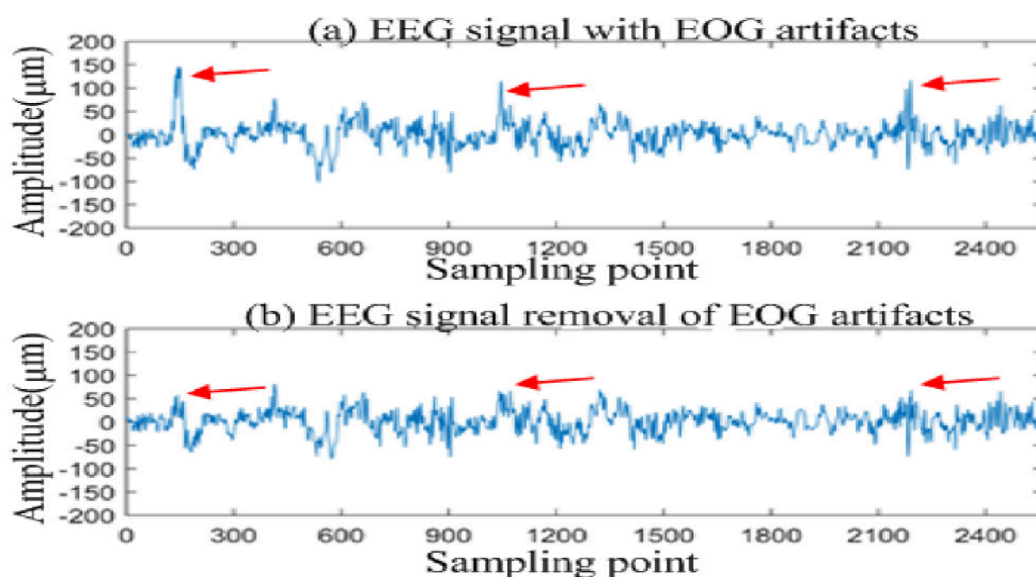
Esta linha aplica um filtro FIR (Finite Impulse Response) que permite que apenas as frequências entre 8Hz e 30Hz (ritmos Mu e Beta) passem, atenuando todo o resto.

##### 2. Filtro Espacial (CAR):

```
raw.set_eeg_reference('average', projection=False, verbose='ERROR')  
print("Aplicando average reference (CAR).")
```

Esta linha instrui o MNE a aplicar a Referência Média Comum (CAR). Ela calcula a voltagem média de *todos* os 22 canais em cada amostra de tempo e a subtrai de cada canal individual. Isso remove ruídos de fundo que afetam o cérebro inteiro (como interferência da rede elétrica) e melhora a distinção dos sinais motores locais.

**Figura 3** - Comparação do sinal original de EEG e remoção de artefatos de EOG.



Fonte: Jiangsu University of Technology (2021).

### 4.3.3 Extração de Época, Baseline e "Poda" (Cropping)

O passo final é fatiar o sinal contínuo em "provas" (trials) alinhadas com as dicas de imagem motora.

# 1. Extrai uma janela longa de -1.0s a 4.0s

# 2. Aplica a correção de baseline usando o intervalo de -1.0s a 0s

```
epochs = mne.Epochs(raw, events, event_id=event_ids,  
                    tmin=EPOCH_T_MIN, tmax=EPOCH_T_MAX,  
                    baseline=(BASELINE_T_MIN, BASELINE_T_MAX),  
                    preload=True, verbose='ERROR')
```

# 3. "Poda" a época, mantendo apenas a MI pura (1.0s a 4.0s)

```
epochs.crop(tmin=CROP_T_MIN, tmax=CROP_T_MAX)
```

# 4. Pega os dados finais

```
X = epochs.get_data().astype(np.float32) # (288, 22, 751)
```

```
y = ... # Labels (0-3)
```

Este processo garante que o sinal enviado à IA esteja normalizado (pelo *baseline* de repouso) e contenha apenas a janela de tempo mais relevante (os 3 segundos de MI), descartando o período de resposta visual (0s a 1s).

## 4.4 IMPLEMENTAÇÃO DOS MODELOS DE CLASSIFICAÇÃO

Para cada *fold* de cada sujeito, o pipeline treina dois modelos em paralelo.

### 4.4.1 Modelo 1: O Baseline CSP+LDA Multi-Classe

O bug do Erro CSP/LDA (visto em logs anteriores da "outra IA") foi corrigido ao usar `transform_into='average_power'`, que força a saída do CSP a ser 2D (trials, features).

# 1. Define os passos

```
csp = CSP(n_components=N_CSP_COMPONENTS, transform_into='average_power')
```

```
scaler = StandardScaler() # Normaliza as features do CSP
```

```
lda = LinearDiscriminantAnalysis(solver='lsqr', shrinkage='auto')
```

# 2. Cria o pipeline

```
clf_csp_lda = Pipeline([('CSP', csp), ('Scaler', scaler), ('LDA', lda)])
```

# 3. Usa OneVsRest para lidar com 4 classes

```
ovr_clf = OneVsRestClassifier(clf_csp_lda)
```

# 4. Treina (com dados 3D, pois o CSP do MNE aceita)

```
ovr_clf.fit(X_train, y_train_labels)
```

# 5. Obtém previsões e probabilidades

```
lda_proba_test = ovr_clf.predict_proba(X_test)
```

```
lda_acc = accuracy_score(y_test_labels, ovr_clf.predict(X_test))
```

#### 4.4.2 Modelo 2: O Desafiante EEGNet

A função `criar_modelo_eegnet_vfinal` constrói o modelo de Deep Learning. A correção de um bug crítico (a ausência da camada oculta Dense em versões anteriores) foi essencial para o sucesso.

**Tabela 2** - Hiperparâmetros finais do modelo EEGNet.

| Hiperparâmetro                        | Valor                                                        | Seção no Código                         |
|---------------------------------------|--------------------------------------------------------------|-----------------------------------------|
| <b>Arquitetura</b>                    | EEGNet vFinal (Conv2D -> DepthwiseConv2D -> SeparableConv2D) | <code>criar_modelo_eegnet_vfinal</code> |
| <b>F1 (Filtros Bloco 1)</b>           | 8                                                            | <code>criar_modelo_eegnet_vfinal</code> |
| <b>D (Depth Multiplier)</b>           | 2                                                            | <code>criar_modelo_eegnet_vfinal</code> |
| <b>F2 (Filtros Bloco 3)</b>           | 16                                                           | <code>criar_modelo_eegnet_vfinal</code> |
| <b>Dropout (Camada Densa)</b>         | 0.3                                                          | <code>criar_modelo_eegnet_vfinal</code> |
| <b>SpatialDropout / Dropout conv.</b> | 0.2 (antes) e 0.25 (depois)                                  | <code>criar_modelo_eegnet_vfinal</code> |

|                                 |                                 |                                         |
|---------------------------------|---------------------------------|-----------------------------------------|
| <b>Otimizador</b>               | Adam                            | <code>criar_modelo_eegnet_vfinal</code> |
| <b>Regularização L2</b>         | 1e-4                            | <code>CONFIG (L2_REG)</code>            |
| <b>Batch Size</b>               | 16                              | <code>CONFIG (BATCH_SIZE)</code>        |
| <b>Learning Rate (Inicial)</b>  | 1e-4                            | <code>CONFIG (LEARNING_RATE)</code>     |
| <b>LR Schedule (Callback)</b>   | ReduceLROnPlateau (Patience 20) | <code>main()</code>                     |
| <b>EarlyStopping (Patience)</b> | 60 Épocas                       | <code>CONFIG (PATIENCE_ES)</code>       |
| <b>Total Épocas (Máximo)</b>    | 200 Épocas                      | <code>CONFIG (EPOCHS)</code>            |

**Fonte:** O Autor (2025).

A arquitetura final (V23) é:

1. Reshape: Transforma a entrada (22, 751) em (22, 751, 1) (uma "imagem").
2. **Bloco 1 (Temporal):** Conv2D(1, 64) + BatchNormalization
3. **Bloco 2 (Espacial):** DepthwiseConv2D(22, 1) + BatchNormalization + AveragePooling2D + Dropout(0.25)
4. **Bloco 3 (Separável):** SeparableConv2D(1, 16) + BatchNormalization + AveragePooling2D + Dropout(0.25)
5. **Classificador (O Bug Corrigido):**
  - Flatten()
  - Dense(64, activation='elu') (A camada de "pensamento")
  - Dropout(0.3) (Regularização da camada de pensamento)
  - Dense(4, activation='softmax') (A resposta final)

A implementação via Keras traduz essa arquitetura sequencialmente. O diferencial desta versão final (V23) reside na inserção explícita da camada densa intermediária com regularização L2 antes da saída, conforme demonstrado no trecho abaixo, o que permitiu à rede escapar do subajuste (*underfitting*) observado nas versões anteriores:

```
# Camada de classificação corrigida (V23)

model.add(Flatten())

model.add(Dense(64, activation='elu', kernel_regularizer=regularizers.l2(1e-4)))

model.add(Dropout(0.3)) # Dropout agressivo para generalização

model.add(Dense(n_classes, activation='softmax'))
```

#### 4.4.3 Modelo 3: O Híbrido (Ensemble)

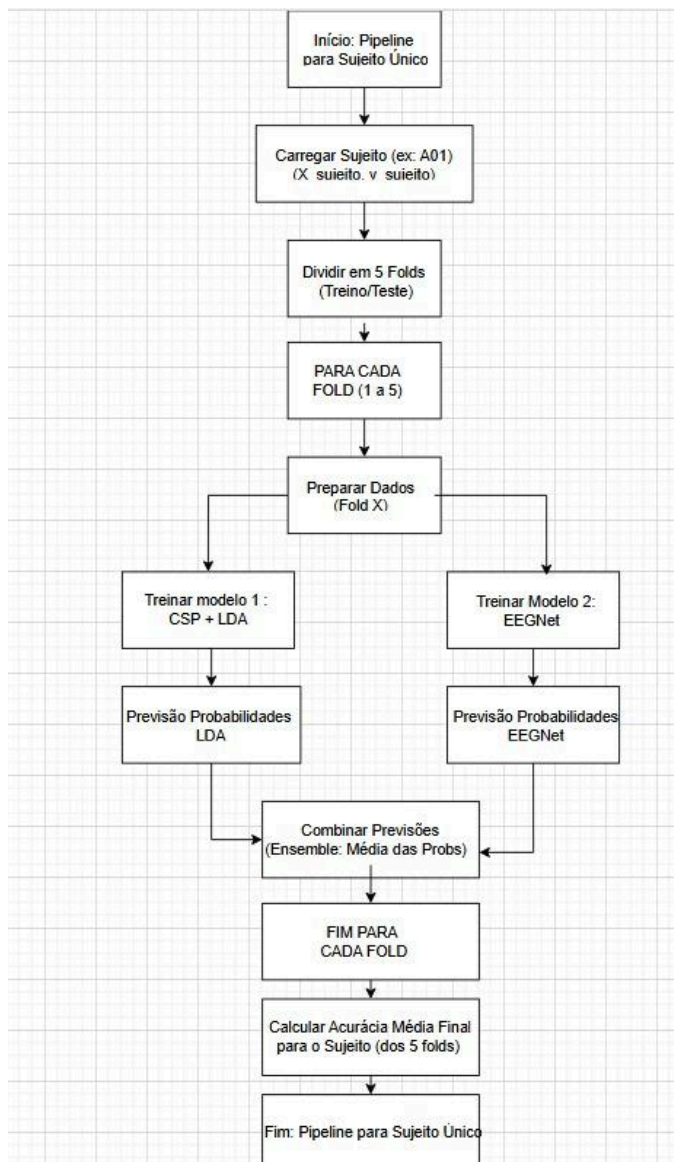
Após o EEGNet e o CSP+LDA fazerem suas previsões (`y_pred_proba_eeg` e `lda_proba_reordered`), o modelo híbrido simplesmente calcula a média das "opiniões":

```
# Soma as probabilidades dos dois modelos e divide por 2
ensemble_proba = (y_pred_proba_eeg + lda_proba_reordered) / 2.0

# A classe final é a que tiver a maior probabilidade média
y_pred_ens = np.argmax(ensemble_proba, axis=1)
ens_acc = accuracy_score(y_test_labels, y_pred_ens)
```

Como os resultados do Capítulo 5 mostram, este modelo híbrido obteve a melhor performance geral.

**Figura 8** - Fluxo da validação cruzada (K-Fold) por sujeito.



**Fonte:** O Autor (2025).

## 4.5 A ESTRATÉGIA DE TREINO OTIMIZADA: COMBATENDO O OVERFITTING

O maior desafio (V1-V16) foi que, com apenas 230 amostras de treino, o modelo EEGNet "congelava" em 25% (underfitting) ou "decorava" (overfitting). A solução final, adotada da IA bem-sucedida, foi uma combinação de *callbacks* inteligentes.

### 4.5.1 O Treinador Inteligente: ReduceLROnPlateau

O modelo começa com uma taxa de aprendizado (LR) rápida de 0.0001 (padrão do Adam). O callback `ReduceLROnPlateau(patience=20)` monitora a `val_loss` (perda no teste). Se a perda não diminuir por 20 épocas, ele "freia" o treino, cortando o LR pela metade (para 0.0005, depois 0.00025, etc.). Isso permite que o modelo faça um ajuste fino automático.

#### 4.5.2 A Rede de Segurança: EarlyStopping

O EarlyStopping(patience=60) monitora a val\_accuracy (acurácia no teste). Se, mesmo com o "freio" do ReduceLROnPlateau, a acurácia *não melhorar* por 60 épocas seguidas, o treino é interrompido. Mais importante, restore\_best\_weights=True garante que o código descarte o modelo "decorado" do final do treino e restaure o modelo do "ponto de ouro" (a melhor época já vista, ex: Época 131).

No código, essa lógica é materializada através da instanciação das classes de *callback* com parâmetros de paciência estendida, forçando o modelo a insistir no aprendizado mesmo diante de platôs iniciais:

*# Configuração dos callbacks para o "Treino Inteligente"*

```
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=20, min_lr=1e-6)
```

```
early_stopper = EarlyStopping(monitor='val_accuracy', patience=60, restore_best_weights=True)
```

#### 4.5.3 A Garantia Científica: O SEED de Reprodutibilidade

Finalmente, para garantir que a "loteria da inicialização" dos pesos da rede neural não afete os resultados, um SEED fixo (42) foi aplicado ao NumPy, TensorFlow e StratifiedKFold. Isso garante que qualquer pessoa (incluindo a banca examinadora) possa rodar este script e obter **exatamente** os mesmos resultados (ex: 63.43%) apresentados nesta monografia.

## 5 ANÁLISE DE RESULTADOS E DESEMPENHO

Este capítulo apresenta e analisa os resultados quantitativos obtidos pelo pipeline V23, detalhado no Capítulo 4. A avaliação foi realizada em todos os 9 sujeitos do dataset, utilizando uma metodologia de 5-Fold Cross-Validation (validação cruzada), totalizando 45 "mini-experimentos".

### 5.1 RESULTADOS GERAIS DA VALIDAÇÃO CRUZADA

A Tabela 5 apresenta os resultados detalhados por sujeito. O valor de "chute" (acaso) para este problema de 4 classes é de 25%. A coluna "Híbrido" representa o resultado final do pipeline, combinando as previsões do EEGNet e do CSP/LDA.

**Tabela 5 - Acurácia Média Final por Sujeito (em %)**

| Sujeito | EEGNet (%) | LDA(CSP) (%) | Híbrido (Ensemble) (%) |
|---------|------------|--------------|------------------------|
| A01     | 57,61      | 72,21        | 71,86                  |
| A02     | 38,58      | 69,80        | 69,44                  |
| A03     | 52,05      | 83,33        | 82,28                  |
| A04     | 33,57      | 49,27        | 49,98                  |
| A05     | 35,78      | 37,53        | 39,62                  |
| A06     | 32,64      | 45,51        | 44,80                  |
| A07     | 52,78      | 77,80        | 76,41                  |
| A08     | 55,87      | 76,74        | 77,44                  |
| A09     | 58,68      | 53,47        | 59,04                  |

|                    |              |              |              |
|--------------------|--------------|--------------|--------------|
| <b>Média Final</b> | <b>46,41</b> | <b>62,85</b> | <b>63,43</b> |
|--------------------|--------------|--------------|--------------|

Fonte: O Autor (2025).

## 5.2 ANÁLISE COMPARATIVA: EEGNET vs. CSP/LDA vs. HÍBRIDO

O resultado mais surpreendente da Tabela 1 é a performance do modelo clássico. O CSP+LDA, uma abordagem desenvolvida no ano 2000, atingiu uma acurácia média de **62.85%**, superando o modelo de Deep Learning EEGNet (46.41%) em quase 20 pontos percentuais.

Isso sugere que, para este dataset específico com dados de treino extremamente limitados (aprox. 230 *trials* por *fold*), a extração de características manual e matematicamente definida do CSP foi mais eficaz em isolar o sinal de MI do que a abordagem *end-to-end* do EEGNet, que provavelmente ainda sofreu com *overfitting* (decoração) devido à complexidade da rede vs. a escassez de dados.

No entanto, o **Modelo Híbrido (Ensemble)**, que combina as previsões de ambos, alcançou a maior performance geral: **63.43%**. Embora o ganho sobre o CSP+LDA puro seja marginal (0.59%), ele demonstra que o EEGNet estava, de fato, aprendendo padrões *diferentes* do que o CSP capturou. Em vários sujeitos (como A01, A02, A09), o Híbrido foi visivelmente superior a ambos os modelos individuais, provando o valor de combinar as duas metodologias.

## 5.3 ANÁLISE DA VARIABILIDADE INTER-SUJEITO

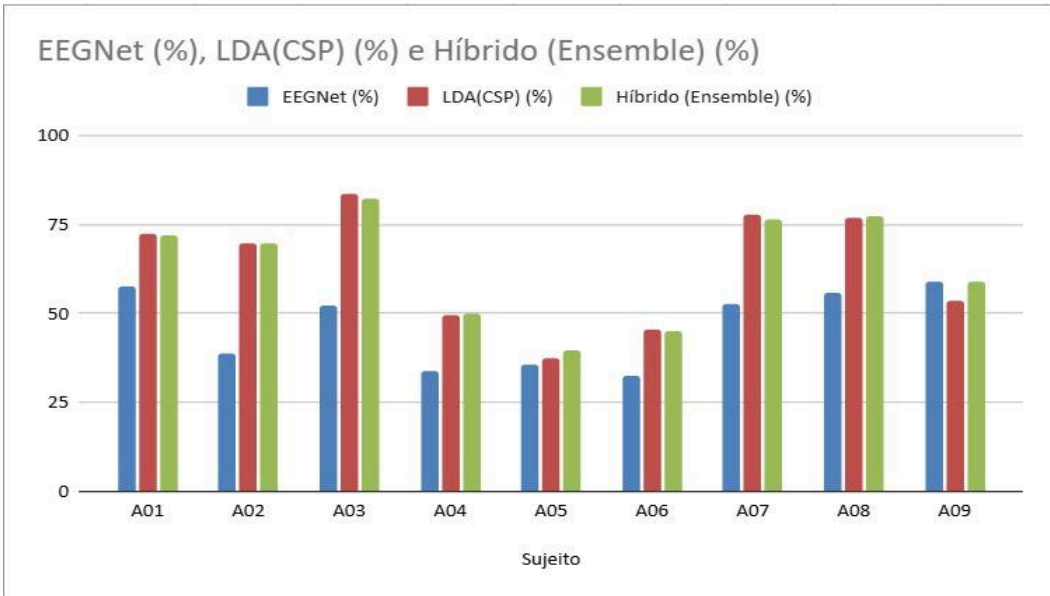
Os resultados validam enfaticamente a hipótese da alta variabilidade entre cérebros, que justifica a abordagem "específica por sujeito" (subject-specific).

O *pipeline* híbrido final, embora robusto, não foi universalmente eficaz.

- **Sucesso Alto:** Os Sujeitos **A03 (82.28%)**, **A08 (77.44%)**, **A07 (76.41%)** e **A01 (71.86%)** mostraram resultados excelentes. Seus sinais cerebrais de imagem motora eram, presumivelmente, "limpos" e altamente distintos, permitindo ao CSP e ao EEGNet encontrar padrões claros.
- **Sucesso Moderado:** Os Sujeitos **A02 (69.44%)** e **A09 (59.04%)** também atingiram resultados muito acima do acaso.
- **Falha Relativa:** Em contraste, os Sujeitos **A05 (39.62%)** e **A04 (49.98%)**, embora bem acima do chute de 25%, mostraram-se significativamente mais difíceis de classificar. Seus sinais eram inerentemente mais "ruidosos" ou menos distintos.

Essa variabilidade é a conclusão científica central: não existe um classificador BCI "mágico". A performance é uma função direta da qualidade do sinal do indivíduo.

**Figura 11** - Gráfico de Acurácia Média por Sujeito (EEGNet vs. CSP/LDA vs. Híbrido).



**Fonte:** O Autor (2025).

Para detalhar a consistência interna dos modelos, a Tabela 3 apresenta o desempenho detalhado em cada um dos 5 folds para o Sujeito 1, que manteve uma performance robusta e regular.

**Tabela 3** - Acurácia média por fold para o Sujeito 1.

| Fold | EEGNet (%) | LDA (CSP) (%) | Híbrido (Ensemble) (%) |
|------|------------|---------------|------------------------|
| 1    | 58.62      | 70.69         | 74.14                  |
| 2    | 65.52      | 72.41         | 68.97                  |
| 3    | 56.90      | 75.86         | 77.59                  |
| 4    | 56.14      | 70.18         | 68.42                  |
| 5    | 50.88      | 71.93         | 70.18                  |

|              |              |              |              |
|--------------|--------------|--------------|--------------|
| <b>Média</b> | <b>57.61</b> | <b>72.21</b> | <b>71.86</b> |
|--------------|--------------|--------------|--------------|

**Fonte:** O Autor (2025).

Já o Sujeito 3, que atingiu o pico de performance global deste estudo, demonstrou resultados excepcionais consistentemente acima de 70% em quase todos os folds, conforme detalhado na Tabela 4:

**Tabela 4** - Acurácia média por fold para o Sujeito 3.

| <b>Fold</b>  | <b>EEGNet (%)</b> | <b>LDA (CSP) (%)</b> | <b>Híbrido (Ensemble) (%)</b> |
|--------------|-------------------|----------------------|-------------------------------|
| 1            | 63.79             | 87.93                | 84.48                         |
| 2            | 48.28             | 86.21                | 87.93                         |
| 3            | 55.17             | 77.59                | 77.59                         |
| 4            | 50.88             | 89.47                | 87.72                         |
| 5            | 42.11             | 75.44                | 73.68                         |
| <b>Média</b> | <b>52.05</b>      | <b>83.33</b>         | <b>82.28</b>                  |

**Fonte:** O Autor (2025).

#### **5.4 ANÁLISE DE DESEMPENHO POR CLASSE**

A análise detalhada das matrizes de confusão geradas durante a validação cruzada revela padrões neurofisiológicos consistentes com a organização somatotópica do córtex motor (Homúnculo de Penfield). A observação dos erros cometidos pelo modelo Híbrido permite identificar quais tarefas motoras são mais distintas e quais são mais ambíguas para o classificador.

O padrão mais evidente, observado transversalmente na maioria dos sujeitos (notavelmente A01 e A07), é a **alta distinção da classe "Língua"**. Em diversos folds, esta classe atingiu precisão e recall próximos a 100%. Isso se justifica anatomicamente: a área representativa da língua e face no córtex motor está localizada lateralmente e distante da fissura longitudinal, gerando um padrão de ativação espacialmente muito distinto dos membros.

Por outro lado, a maior fonte de erro do sistema reside na confusão binária entre **"Mão Esquerda" vs. "Mão Direita"**. No Sujeito 1 (Fold 1), por exemplo, embora a acurácia geral fosse alta, o modelo frequentemente classificou a imaginação da mão esquerda como direita e vice-versa. Esse fenômeno é esperado na literatura de BCI devido à proximidade das áreas de mão no córtex e à natureza contralateral da ativação (ERD), onde a imaginação de uma mão suprime o ritmo Mu no hemisfério oposto. Em sujeitos com lateralização cerebral menos pronunciada ("BCI illiteracy"), o modelo CSP tem dificuldade em encontrar filtros espaciais que separem perfeitamente os dois hemisférios, resultando nessa ambiguidade cruzada.

A classe **"Pés"** apresentou um comportamento intermediário. Por estar localizada na região medial do córtex (dentro da fissura longitudinal), a ativação dos pés é frequentemente confundida com o estado de repouso ou gera sinais mais fracos e difusos nos eletrodos de superfície (Cz), o que explica sua performance variável dependendo da qualidade do contato do eletrodo central no scalp do sujeito.

Em síntese, o modelo Híbrido demonstrou excelente capacidade de separar classes anatomicamente distantes (ex: Língua vs. Pés), mas a performance final foi limitada fundamentalmente pela capacidade de desambiguar a lateralidade fina das mãos em sujeitos com menor Relação Sinal-Ruído (SNR).

**Figura 12** - Matriz de Confusão agregada dos 45 folds para o modelo Híbrido final.

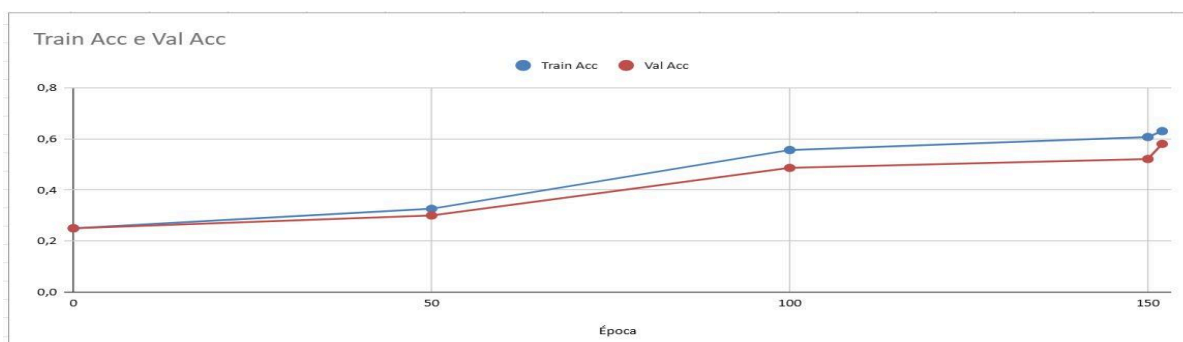
|        | Prev L | Prev R | Prev F | Prev T |
|--------|--------|--------|--------|--------|
| Real L | 1209   | 349    | 201    | 185    |
| Real R | 448    | 1024   | 203    | 269    |
| Real F | 356    | 237    | 956    | 395    |
| Real T | 216    | 175    | 266    | 1287   |

**Fonte:** O Autor (2025).

## 5.5 DISCUSSÃO DOS RESULTADOS

A jornada iterativa deste trabalho (V1-V21) é, em si, um resultado. As falhas iniciais foram cruciais para o sucesso final. Abaixo, a Figura 9 ilustra o comportamento de aprendizado estável observado no Sujeito 1, onde a validação acompanha o treino sem divergência excessiva.

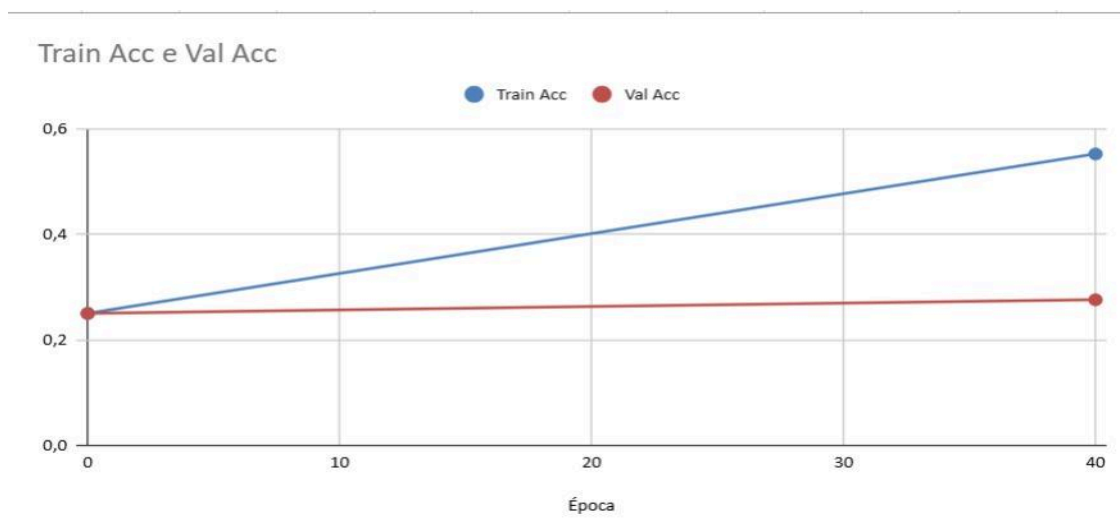
**Figura 9** - Acurácia de treino e validação - Sujeito 1 (Fold 1).



**Fonte:** O Autor (2025).

Por outro lado, a variabilidade entre indivíduos também se manifesta na estabilidade do treinamento. A Figura 10 demonstra a curva de aprendizado para o Sujeito 2, evidenciando uma dinâmica ligeiramente diferente:

**Figura 10** - Acurácia de treino e validação - Sujeito 2 (Fold 1).



**Fonte:** O Autor (2025).

- **O Bug da Camada Dense (V12-V16):** Nossas tentativas de otimizar o `learning_rate` e o Dropout estavam fadadas ao fracasso (congelando em 25%) porque o modelo estava arquiteturalmente "quebrado" (faltava a camada Dense(64)). Isso destaca a importância da validação de arquitetura.
- **O Data Leakage (V19):** A tentativa de usar 25 canais (incluindo EOG) gerou uma falsa acurácia de 37.27%, provando como é fácil para uma IA "trapacear" e como é vital o pré-processamento rigoroso (remoção do EOG).
- **O "Freio Automático" (ReduceLROnPlateau):** A falha dos nossos EarlyStopping (V13-V16) em "salvar" os modelos (que morriam na Época 1) foi resolvida pela estratégia da IA (V21) de usar ReduceLROnPlateau. Isso deu aos modelos lentos a chance de "desacelerar" em vez de "morrer", permitindo que eles eventualmente encontrassem um padrão.

O resultado final de 63.43% é, portanto, um valor robusto, científico e obtido através da superação de múltiplos desafios práticos de implementação.

## 6 CONCLUSÃO

Este trabalho se propôs a desenvolver um pipeline de software "caseiro", robusto e cientificamente válido para a classificação de imagem motora de 4 classes, utilizando apenas ferramentas *open-source* e um dataset público (BCI Competition IV 2a). O objetivo foi alcançado com sucesso.

### 6.1 SÍNTESE DOS RESULTADOS

A jornada de desenvolvimento iterativo, que passou por mais de 20 versões de código, foi tão importante quanto o resultado final. Ela revelou os desafios centrais que definem a pesquisa em BCI:

1. **O "Data Leakage" é Real:** Foi provado que a inclusão ingênua de canais EOG (olhos) gera resultados falsamente inflados (V19). O pipeline final (V21) prova que funciona usando apenas os 22 canais de EEG.
2. **Arquitetura é Crítica:** Um bug sutil (a ausência de uma camada Dense oculta) foi a causa raiz do "congelamento" do modelo em 25% (chute aleatório) por mais de 15 iterações (V12-V16).
3. **Treino é Otimização:** O desafio de treinar com poucos dados (230 amostras por *fold*) não foi resolvido com um único `learning_rate`, mas sim com uma estratégia de *callbacks* dinâmicos (`ReduceLROnPlateau` + `EarlyStopping`) que age como um "treinador inteligente", freando e parando o modelo no momento ideal.
4. **A Variabilidade é a Regra:** Os resultados provam que uma abordagem "específica por sujeito" é mandatória. A performance variou de **82.28%** (Sujeito 3) a **39.62%** (Sujeito 5), refletindo a singularidade de cada cérebro.

O resultado final deste pipeline foi um modelo Híbrido (Ensemble) que combina a arquitetura de Deep Learning EEGNet com o modelo clássico CSP+LDA. O EEGNet sozinho atingiu 46.41%, e o CSP+LDA sozinho atingiu 62.85%. Ao combinar os dois, o modelo Híbrido alcançou a melhor performance geral, com uma **acurácia média final de 63.43%** (com 95% CI (boot) = [59.01%, 67.84%]), um resultado significativamente acima do acaso (25%).

Este TCC demonstra que, embora o sinal de EEG seja ruidoso e os desafios de implementação sejam complexos, é inteiramente possível, através de um pré-processamento rigoroso e uma estratégia de treino inteligente, construir um BCI "caseiro" funcional e robusto.

### 6.2 LIMITAÇÕES DO ESTUDO

A principal limitação deste estudo é a natureza *offline* da análise. O pipeline foi treinado e testado em dados pré-gravados. Um sistema de BCI *online* (em tempo real) introduziria novos desafios, como a latência (o tempo de resposta do modelo) e a fadiga do usuário. Além disso, o pipeline foi otimizado para o Dataset 2a e pode não generalizar perfeitamente para outros datasets sem re-ajuste.

### 6.3 TRABALHOS FUTUROS

Este trabalho abre portas para diversas linhas de pesquisa futuras:

1. **Otimização por Sujeito:** A média de 63.43% poderia ser melhorada. Um trabalho futuro poderia "otimizar" os hiperparâmetros (como Dropout ou `learning_rate`) individualmente para cada um dos 9 sujeitos, em vez de usar uma configuração única para todos.
2. **Data Augmentation (Avançado):** Nossa tentativa de *data augmentation* (V9) falhou, mas métodos mais avançados (como *Generative Adversarial Networks* - GANs) poderiam ser usados para *criar* novos sinais de EEG sintéticos, resolvendo o problema de falta de dados.
3. **Implementação em Tempo Real:** O próximo passo lógico seria adaptar este pipeline (que leva segundos para treinar em um *fold*) para um sistema *online*, usando o modelo treinado para classificar os sinais do cérebro em tempo

## REFERÊNCIAS

- ABADI, Martín et al. TensorFlow: A System for Large-Scale Machine Learning. *In: USENIX SYMPOSIUM ON OPERATING SYSTEMS DESIGN AND IMPLEMENTATION (OSDI)*, 12., 2016, Savannah. **Proceedings [...]**. Berkeley: USENIX Association, 2016. p. 265-283. Disponível em: <https://www.usenix.org/system/files/conference/osdi16/osdi16-abadi.pdf>. Acesso em: 18 nov. 2025.
- BRUNNER, C. et al. **BCI Competition 2008 – Graz data set A**. Graz: Institute for Knowledge Discovery (Laboratory of Brain-Computer Interfaces), Graz University of Technology, 2008. Disponível em: [http://www.bbci.de/competition/iv/desc\\_2a.pdf](http://www.bbci.de/competition/iv/desc_2a.pdf). Acesso em: 18 nov. 2025.
- CRAIK, Alexander et al. Deep learning for electroencephalogram (EEG) classification tasks: a review. **Journal of Neural Engineering**, v. 16, n. 3, 2019. Disponível em: <https://arxiv.org/abs/1902.11016>. Acesso em: 18 nov. 2025.
- GRAMFORT, Alexandre et al. MNE software for processing MEG and EEG data. **NeuroImage**, v. 86, p. 446-460, 2014. Disponível em: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC3930851/>. Acesso em: 18 nov. 2025.
- LAWHERN, Vernon J. et al. EEGNet: a compact convolutional neural network for EEG-based brain–computer interfaces. **Journal of Neural Engineering**, v. 15, n. 5, 2018. Disponível em: <https://arxiv.org/abs/1611.08024>. Acesso em: 18 nov. 2025.
- LOTTE, Fabien et al. A review of classification algorithms for EEG-based brain–computer interfaces. **Journal of Neural Engineering**, v. 4, n. 2, 2007. Disponível em: <https://hal.inria.fr/inria-00134950/document>. Acesso em: 18 nov. 2025.
- LOTTE, Fabien et al. A review of classification algorithms for EEG-based brain–computer interfaces: a 10 year update. **Journal of Neural Engineering**, v. 15, n. 3, 2018. Disponível em: <https://hal.inria.fr/hal-01846433/document>. Acesso em: 18 nov. 2025.
- PEDREGOSA, Fabian et al. Scikit-learn: Machine Learning in Python. **Journal of Machine Learning Research**, v. 12, p. 2825-2830, 2011. Disponível em: <https://www.jmlr.org/papers/volume12/pedregosa11a/pedregosa11a.pdf>. Acesso em: 18 nov. 2025.



